그래프 표현학습 (Graph Representation Learning)

무작위 걷기와 정상분포 · Node2Vec: 걷기에 skip-gram · PageRank: 정상분포를 중요도로 · 확장: 단백질·금융·교통

Machine Learning 1 · 13주차

한국과학영재학교 (KSA)

Week 13

이번 주차 개요

- **13.1 Random Walk** — 무작위 걷기 = 마르코프 체인, 머무는 비율 $\propto$ 도
- **13.2 Node2Vec** — 걷기 = 문장, skip-gram 학습, $p \cdot q$ 탐색 스타일
- **13.3 PageRank** — 댐핑 + 순간이동, 거듭제곱법 손계산
- **13.4 확장** — 단백질(DeepFRI) · 금융(AML) · 교통(STGCN)

그래프로 표현된 데이터를, 신경망이나 통계 모델이 다룰 수 있는 벡터로 어떻게 바꾸는가?

13.1 Random Walk 기초

이 장은 무엇을 묻는가 — AlphaFold가 준 관점

- 핵심 관점 중 하나: 단백질을 **그래프**로 본다.
- 각 아미노산(잔기, residue) = **노드**, 공간적으로 가까운 잔기 쌍 사이의 관계 = **엣지**.

그래프로 표현된 데이터를 벡터로 어떻게 바꾸는가?

핵심 실: word2vec 스킵그램의 그래프 재사용

14.2절의 스킵그램 원리

"문맥 안에서 **함께 나타나는** 단어는 **비슷한 벡터**를 갖는다"

- "문장"의 역할은 그래프 위의 **무작위 걷기**(random walk).

그래프 위 무작위 걷기 = 마르코프 체인

정의

노드 집합 $\{0, \ldots, N-1\}$의 그래프 위에서 "지금 서 있는 노드"를 **상태**로 보고, 매 단계에서 **지금 노드의 이웃 중 하나를 골라 이동**하는 확률과정 = **마르코프 체인**(Markov chain).

- 상태 i에서 j로 건너가는 **전이확률** (transition probability):

$$P(i \to j) = \frac{1}{\deg(i)} \quad ((i,j) \in \mathcal{E}), \qquad P(i \to j) = 0 \quad (\text{그 외})$$

같은 체인, 다른 파라미터 — 다음 두 절의 예고

- **13.2 Node2Vec**: 이 전이확률에 p, q 두 계수를 붙여 **편향**(bias)시키는 변형.
- **13.3 PageRank**: 이 전이확률에 "**순간이동**"을 섞은 변형.

한 줄 정리

"**같은 마르코프 체인** 위에 파라미터만 바꾸는 것" — **같은 무작위 걷기에 서로 다른 질문을 던진다.**

마르코프 체인의 핵심 성질: 무자기성

다음 상태를 결정하는 것은 “지금 어디에 있는지”뿐이다 — 그전에 어떤 경로를 거쳐 왔는지는 **아니다**.

$$P(x_{t+1} \mid x_t, x_{t-1}, \ldots, x_0) = P(x_t \to x_{t+1})$$

상태 분포를 계속 곱하면 — 정상분포

- 전이확률 행렬 P를 "각 노드에 서 있을 확률"인 상태 분포 π에 계속 곱한다:

$$\pi_{t+1} = \pi_t P$$

- 분포는 결국 멈추는 **정상분포**(stationary distribution) π^* 에 도달한다:

$$\pi^* P = \pi^*$$

같은 무작위 걷기, 다른 질문

13.2 Node2Vec

- “이 걷기가 만들어내는 **노드의 순서(시퀀스)**는 어떤 구조를 담고 있는가?”
- 그 시퀀스를 “문장” 취급해 word2vec의 skip-gram을 학습 $\rightarrow$ **노드 임베딩**.

13.3 PageRank

- “이 걷기를 영원히 돌리면 **각 노드에 머무는 확률** π^*는 얼마인가?”
- 그 정상분포 자체를 노드의 “**중요도**”로 읽는다.

한 줄

같은 재료(그래프, 무작위 걷기)에서 하나는 **시퀀스**를 뽑아내고, 하나는 **정상분포**를 뽑아낸다 — 벡터와 스칼라, 두 가지 다른 표현.



Jure Leskovec (Stanford) — Node2Vec 논문 공저자 (Grover & Leskovec, KDD 2016)

왜 “무작위 걷기”가 “문장”을 대신하는 자연스러운 선택인가

- word2vec에서 문장의 본질
- 그래프 노드의 함께 나타남
- 무작위 걷기가 순서를 만든다

머무는 비율은 도(degree)에 비례한다

핵심 성질

무방향 그래프에서 일괄적 무작위 걷기를 무한히 계속하면, 노드 i에 머무는 장기 비율은 그 노드의 **도**(degree, 연결된 엣지 수)에 비례:

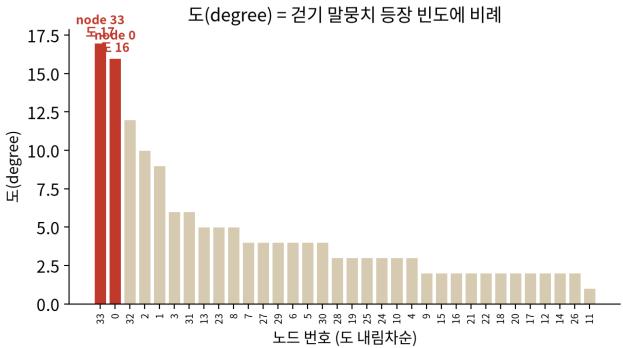
$$\pi(i) = \frac{\deg(i)}{2\,|\mathcal{E}|}$$

($|\mathcal{E}|$ = 엣지 수, $2|\mathcal{E}|$ = 모든 노드의 도의 합)

- 즉 **허브 노드는 걷기 말뭉치에 더 자주 등장** $\rightarrow$ skip-gram 학습에서 **더 많이 업데이트**.

허브는 더 많이 “나타나는 단어”

- 카라테 클럽에서: node 33(관장) 도 17, node 0(감독) 도 16 — 두 파벌 리더가 구조적으로 가장 많이 “나타나는” 단어.
- 임베딩 학습이 자동으로 **구조적으로 중요한 노드에 더 많은 “학습 예산”**을 배정해준다고 볼 수 있다.



주석 — 13.3과의 연결

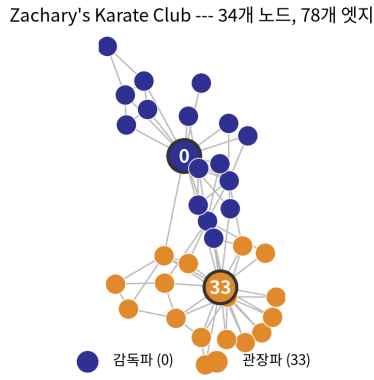
“영원히 돌아다닌 뒤 머무는 비율”은 댐핑을 빼면 곧바로 **PageRank** 그 자체.

Zachary’s Karate Club — 표준 테스트 데이터셋

- 34개 노드, **78개**의 친분 관계(edge)뿐인 작은 그래프.
- 그래프 임베딩이 “제대로 동작하는지” 확인하는 **표준 벤치마크**로 지금까지도 사용.

성공의 기준

라벨(파벌 소속) 없이 순수하게 그래프 구조만 보고 임베딩했는데도 두 파벌이 갈라지면 성공.



코드: 카라테 클럽 엣지 목록

```
KARATE_EDGES = [
    (0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(0,7),(0,8),(0,10),(0,11),
    (0,12),(0,13),(0,17),(0,19),(0,21),(0,31),(1,2),(1,3),(1,7),(1,13),
    (1,17),(1,19),(1,21),(1,30),(2,3),(2,7),(2,8),(2,9),(2,13),(2,27),
    (2,28),(2,32),(3,7),(3,12),(3,13),(4,6),(4,10),(5,6),(5,10),(5,16),
    (6,16),(8,30),(8,32),(8,33),(9,33),(13,33),(14,32),(14,33),(15,32),
    (15,33),(18,32),(18,33),(19,33),(20,32),(20,33),(22,32),(22,33),
    (23,25),(23,27),(23,29),(23,32),(23,33),(24,25),(24,27),(24,31),
    (25,31),(26,29),(26,33),(27,33),(28,31),(28,33),(29,32),(29,33),
    (30,32),(30,33),(31,32),(31,33),(32,33),
]
```

무방향 그래프: 친분 관계 78개. node 0 = 감독, node 33 = 관장.

코드: 일괄적 무작위 걷기로 말뭉치 만들기

```python
def random_walk(neighbors, start, length):
    walk = [start]
    for _ in range(length - 1):
        cur = walk[-1]
        if not neighbors[cur]:
            break
        walk.append(random.choice(neighbors[cur]))
    return walk

def build_walks(edges, n_nodes, walks_per_node=10, walk_length=8):
    neighbors = {i: [] for i in range(n_nodes)}
    for a, b in edges:
        neighbors[a].append(b)
        neighbors[b].append(a)
    walks = []
    for node in range(n_nodes):
        for _ in range(walks_per_node):
            walks.append([str(n) for n in random_walk(neighbors, node,
                                                      walk_length)])
    return walks
```

`random_choice` = 이웃을 동등한 확률로 고르는 일괄적 걷기. 각 걷기 = “문장”, 각 노드 번호 = “단어”.

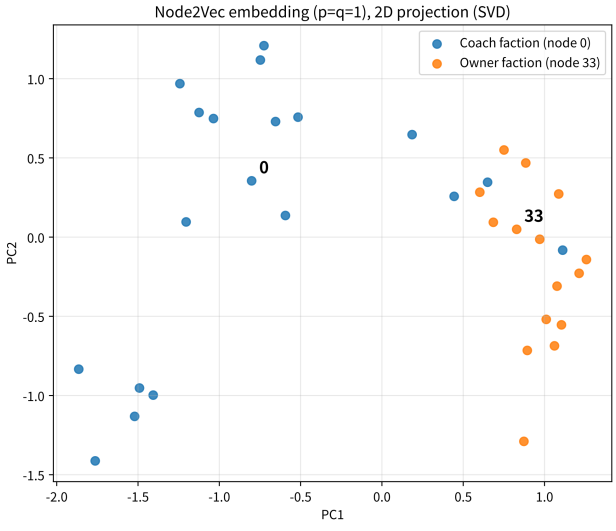
걷기 → 말뭉치 → skip-gram → 임베딩

1. **걷기** — `build_walks` 로 34개 노드 × 노드당 10개 걷기(길이 8) 생성.
2. **시각화** — 14.1절의 **PCA**로 2차원까지 압축해 그리면, 실제로 두 파벌이 공간적으로 갈라진다.

같은 아이디어의 두 얼굴

word2vec과 Node2Vec은 서로 다른 데이터(텍스트 vs. 그래프)에 적용됐을 뿐, "**함께 자주 나타나는 것들은 비슷한 벡터를 갖도록 학습한다**"는 **같은 아이디어의 두 얼굴**.

결과: 라벨 없이 본 임베딩이 분열 구조를 회복



$p{=}q{=}1$ (일괄적 걷기)의 걷기 34×10개(길이 8)로 Node2Vec 학습($d{=}8$차원)한 임베딩을 SVD로 2차원 투영. 파랑 = 감독파, 주황 = 관장파. 두 군집이 2차원 공간에서 상당히 선명하게 갈라진다.

라벨을 전혀 보여주지 않고 구조만 본 임베딩이, 1977년의 실제 분열을 “공간적으로” 회복했다.

해석: 리더는 “경계”가 아니라 군집 중심에

- 두 파벌이 한쪽/다른 쪽으로 갈라지고, 두 파벌 리더 (node 0, node 33)이 **각 군집의 중심 부근**에 있다.
- “리더는 두 파벌의 **경계**에 있어야 할 것” 같은 직관과 달리, 실제로는 군집 **내부에 잘 박혀 있다**.

13.2 Node2Vec: 걷기에 skip-gram을 그대로

13.1을 한 줄로: 걷기 안의 인접 = “함께 나타남”

확립된 것

그래프 위의 **무작위 걷기**는 마르코프 체인이고, 그 걷기 안의 **인접 노드들은 서로를 “함께 나타남”으로 잇는다**.

- 이제 그 “함께 나타남의 순서”, 곧 **걷기 시퀀스**를 14.2절 word2vec의 **skip-gram** 학습에 **그대로** 넣으면 된다.

(Grover & Leskovec, 2016)

random walk로 생성한 경로들에 word2vec의 skip-gram을 **그대로 학습시킨 것**.

word2vec의 세 요소를 그래프 버전으로 치환

word2vec	→	Node2Vec
문장	→	한 노드에서 출발해 이웃을 무작위로 따라가며 만든 **걷기 경로**(노드 시퀀스)
단어	→	그 경로 위에 순서대로 나오는 **노드**
학습	→	14.2절의 `train_skipgram`을 **한 줄도 고치지 않고** 그대로 (토큰이 단어 대신 노드 번호일

"비슷한 문맥에 나오는 단어는 비슷한 벡터를 갖는다" → "**비슷한 이웃 구조를 가진 노드는 비슷한 벡터를 갖게 된다**"는 문장으로 바뀐 것뿐.

학습이 끝나면: 은닉층 가중치 = 노드 임베딩

- 14.2절과 똑같이 **은닉층 가중치 행이 각 노드의 임베딩 벡터**.

핵심 재료가 신경망을 전제로 하지 않는다

"걷기로 시퀀스를 만들고 그 위에서 함께-나타남을 학습한다"는 틀 자체는 **신경망 없이도 성립**. skip-gram을 쓰든, 더 단순한 "함께 나타난 횟수" 카운트 기반의 선형 분해 (14.2절의 LSI 같은)를 쓰든 — Node2Vec은 그 **더 일반적인 틀** 위에 14.2절의 skip-gram이라는 하나의 실용적 학습자를 골랐을 뿐.

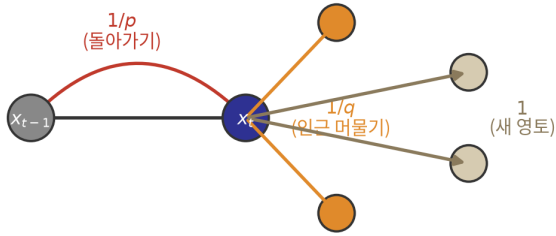
Node2Vec의 진짜 기여: 두 계수 p와 q

- **p와 q로 이웃 탐색 스타일 조절**

다음 노드 선택 확률

- **돌아가기:** $1/p$
- **인근 머물기:** $1/q$
- **새 영토:** 1

$x_{t-1} \to x_t$ 다음, 어느 이웃으로? (가중치 $\propto$ 그림의 값)



계수를 움직이면 탐색 스타일이 기울어진다

- $p = q = 1$: 모든 가중치가 1 $\rightarrow$ 정확히 **일괄적 무작위 걷기**.
- $p < 1$ $(1/p > 1)$: 직전 노드로의 **뒤로 감** (backtracking) 확률이 올라간다.
- $q < 1$ $(1/q > 1)$: 방금 탐색한 **로컬 영역 안에 머물기** 확률이 올라간다.
- $q > 1$ $(1/q < 1)$: 이미 본 영역을 다시 밟는 것이 **억제**되어 걷기가 **새 영토로 확장**하려 한다.

원본 논문의 두 표준 프리셋

설정	탐색 스타일	강조하는 구조
$(p{=}1, q{=}16)$ — “node2vec”	**BFS형**(넓이 우선)	**구조적 동치**(structural equivalence) — 역할의 유사성
$(p{=}16, q{=}1)$ — “deep walk”	**DFS형**(깊이 우선)	**동질성**(homophily) — 같은 커뮤니티 속한 것의 유사성

일괄적 무작위 걷기는 둘을 **섞어서** 담고, p, q가 이 중 어느 쪽에 더 초점을 맞추게 하는 “**다이얼**” 인 셈.

동질성 vs 구조적 동치 — 서로 다른 두 유사도

동질성 (homophily)

- 같은 커뮤니티의 노드는 **동질**하다.
- 예: 카라테 클럽에서 **한 파벌의 멤버끼리** 서로 비슷.
- $\rightarrow$ DFS형 $(16, 1)$ 가 강조.

구조적 동치 (structural equivalence)

- **서로 다른** 커뮤니티의 두 노드도 **역할이 같을** 수 있다.
- 예: 다른 부서 두 곳에서 각각 "외부와의 **유일한 연결고리**" 역할을 하는 두 멤버.
- $\rightarrow$ BFS형 $(1, 16)$ 가 강조.

코드: 편향 무작위 걷기 (일괄적 걷기를 대체)

```python
def biased_walk(nb, start, length, p, q, rng):
    """의Node2Vec 편향무작위걷기  . nb: 노드 -> 이웃목록 """
    walk = [start]
    prev = None
    for _ in range(length - 1):
        cur = walk[-1]
        weights = []
        for nxt in nb[cur]:
            if prev is None:
                weights.append(1.0)
            elif nxt == prev:
                weights.append(1.0 / p)      # 돌아가기
            elif nxt in nb[prev]:
                weights.append(1.0 / q)      # 로컬영역머무르기
            else:
                weights.append(1.0)          # 새영토로
        r = rng.random() * sum(weights)      # 가중치에비례해뽑기
        acc, chosen = 0.0, nb[cur][0]
        for nxt, w in zip(nb[cur], weights):
            acc += w
            if r <= acc:
                chosen = nxt
                break
        walk.append(chosen)
        prev, cur = cur, chosen
    return walk
```

작은 실험: p와 q가 임베딩을 얼마나 바꾸는가

- **일괄적** $(1, 1)$ — 일괄적 무작위 걷기
- **BFS형** $(1, 10)$ — 새 영토 확장 강조
- **DFS형** $(10, 1)$ — 로컬 영역 머물기 강조

결과

설정	코사인 실루엣*	node 2의 유사 노드 상위 3개
$(1, 1)$ 일괄적	0.351	7 (0.73), 13 (0.72), 9 (0.72)
$(1, 10)$ BFS형	0.342	9 (0.86), 28 (0.73), 7 (0.72)
$(10, 1)$ DFS형	0.393	12 (0.83), 19 (0.70), 17 (0.65)

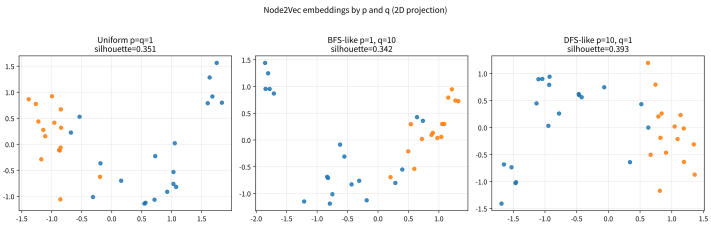
***실루엣**

두 파벌 라벨을 “참 군집”으로 두고, 각 노드가 **자기 파벌 군집의 평균 임베딩**에 상대 파벌 군집의 평균보다 얼마나 가까운지 $-1 \sim +1$ 로 재는 군집 분리 품질 지수 (**1에 가까울수록 잘 분리**).

해석: 방향은 보인다, 크기는 작다

- **DFS형에서 가장 깔끔한 파벌 분리**
- 카라테 클럽에서 파벌 = 커뮤니티이므로, 여기서는 커뮤니티 분리 품질이 곧 “잘 됐다”의 지표.

세 설정을 한 그림에



일괄적 $(1, 1)$, BFS형 $(1, 10)$, DFS형 $(10, 1)$ 으로 각각 학습한 임베딩을 2차원 투영. 파랑 = 감독파, 주황 = 관장파. 세 설정 모두 두 파벌을 나누지만, **DFS형(실루엣 0.393)이 가장 선명하게** 갈라진다.

실용적 교훈

교훈은 이 실험이 아니라 다음 문장

p, q는 "**커뮤니티 구조를 원하느냐, 역할 구조를 원하느냐**"에 맞게 바꾸고, 그 효과를 **downstream 태스크**(분류 · 링크 예측 등)로 **실제 검증**해야 한다.

- 이 실험 자체는 34개 노드의 작은 데모 — 큰 그래프에서 p, q의 효과가 훨씬 크게 벌어진다(원본 논문).

자주 하는 실수: 무방향 엣지를 방향 그대로 넣고 돌리기

- 카라테 그래프는 **무방향**, PageRank는 **방향** 그래프 위에서 정의된다.
- 표준 기법: “친분 관계 1개”를 **양방향 링크 2개**로 바꿔주는 **대칭화** (13.3절의 `directed_edges` 줄).

대칭화 누락의 결과: 감독이 “사라진다”

```
scores_dir = pagerank(KARATE_EDGES, 34)   # 실수: 대칭화누락
# 상위개 5: 33 (0.0759), 32 (0.0280), 31 (0.0135),
#           16 (0.0125), 6 (0.0080)
```

- node 0(감독, 도 16의 **진정한 허브**)이 **상위 5개에서 완전히 사라진다**.

교훈: 도메인 지식과의 교차검증

발견하기 힘든 종류의 버그

대칭화는 **한 줄**이지만, 이 실수는 "**왜 감독이 순위에서 빠졌지?**"라는 **도메인 지식의 교차검증**이 없으면 발견하기 어렵다.

- 습관: 알고리즘 결과와 "알고 있는 사실"이 **충돌하면**, 먼저 **파이프라인의 편의 전제**(여기선 방향성 처리)부터 의심한다.

13.3 PageRank: 정상분포를 중요도로

같은 무작위 걷기, 다른 질문: PageRank

- **무작위 걷기 정상분포를 PageRank로 사용**

같은 재료, 갈라진 결과물

같은 무작위 걷기를 13.2와 이 절이 모두 쓰지만, 묻는 질문이 달라서 나온 결과물이 **임베딩 벡터**와 **스칼라 점수**로 갈라진 것.

1998년: 래리 페이지와 세르게이 브린



Larry Page & Sergey Brin, 2003년 9월
(Google 창업자)

- 웹 링크 구조는 투표 기록
- 링크는 중요도 증거

PageRank의 핵심 아이디어와 정식화

- 그 답 = 그 노드의 "**중요도**" (1998, Brin & Page).
- 많은 페이지가 링크하는 페이지일수록, 또 **중요한 페이지가** 링크하는 페이지일수록, **무작위 서퍼**(random surfer)가 더 자주 머문다.

PageRank 방정식

노드 i의 페이지랭크 $PR(i)$ 정의

$$PR(i) = \frac{1-d}{N} + d \sum_{j \to i} \frac{PR(j)}{\text{outdeg}(j)}$$

- N = 전체 노드 수.
- d **(보통 0.85) = 댐핑 팩터**

재귀식이라 한 번에 못 풀고 — 거듭제곱법

- **거듭제곱법으로 반복 대입**

형태가 수렴을 보장

“**새 값 = teleport 항 + 옛 값들의 가중 평균**”이라는 방정식의 형태 — 이 형태가 반복법의 수렴을 보장하는 구조이며, 바로 뒤에 온다.

코드: pagerank (거듭제곱법)

```python
def pagerank(edges, n_nodes, d=0.85, iters=100):
    outgoing = {i: [] for i in range(n_nodes)}  # directed: a -> b
    for a, b in edges:
        outgoing[a].append(b)
    outdeg = {i: max(1, len(outgoing[i])) for i in range(n_nodes)}
    incoming = {i: [] for i in range(n_nodes)}  # who points to me?
    for a, b in edges:
        incoming[b].append(a)

    pr = {i: 1 / n_nodes for i in range(n_nodes)}
    for _ in range(iters):
        pr = {i: (1 - d) / n_nodes + d * sum(pr[j] / outdeg[j]
                                             for j in incoming[i])
              for i in range(n_nodes)}
    return pr
```

코드: 대칭화하고 카라테 클럽에 적용

```
# Karate 은Club 무방향그래프이므로 ,
# 친분관계하나를양방향링크로취급한다
directed_edges = KARATE_EDGES + [(b, a) for a, b in KARATE_EDGES]
scores = pagerank(directed_edges, 34)
print(sorted(scores.items(), key=lambda kv: -kv[1])[:3])
# node 33, node 이0 가장높다  -- 실제두허브  "" 노드관장(, 감독)과
# 정확히일치
```

(대칭화, $d{=}0.85$, 100회 반복)

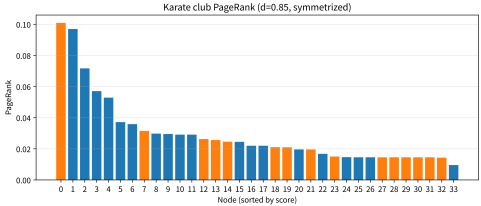
결과: 두 파벌 리더가 정확히 상위 2개

순위	노드	역할	PageRank
1	33	관장	0.1009
2	0	감독	0.0970
3	32	감독 파벌의 연결고리	0.0717

- 라벨을 하나도 안 본 “중요도 순위”가 **역사적 사실** (누가 어떤 파벌이었는지)과 거의 정확히 겹친다.

왜 node 33이 node 0보다 앞서는가

- node 33(**도 17**)이 node 0(**도 16**)보다 앞섬.
- “**무작위 걷기가 더 자주 방문한다**”는 도의 우위가 점수에 **거의 그대로** 드러난 모습.
- 상위권(33, 0, 32)은 모두 두 허브 노드와 그 연결고리.



34개 노드의 PageRank(d=0.85, 대칭화)를 점수 높은 순으로 막대로. 파랑 = 감독파, 주황 = 관장파. 아래로 갈수록 점수가 teleport 하한에 수렴.

손으로 한 번: 노드 4개, 순환 하나, 외딴 끝 하나

- $A \to B, B \to C, C \to A$ — **3-노드 순환**
- $D \to A$ — D는 A만 가리킨다 (들어오는 링크는 **없음**)

teleport 항

$$\frac{1-d}{N} = \frac{0.5}{4} = \mathbf{0.125}$$

1단계 — D는 들어오는 링크가 없다

- D를 가리키는 노드가 없으니 합 $\sum_{j \to D}$ 는 비어 있다:

$$PR(D) = \frac{1-d}{N} = 0.125$$

읽기

D는 **반복을 아무리 돌려도 0.125에 머문다** — “나를 가리키는 이가 없으니 **teleport분밖에** 받을 수 없다”는 뜻.

2단계 — 순환 속 3개 노드에 방정식

$$PR(A) = 0.125 + 0.5\left(\frac{PR(C)}{1} + \frac{PR(D)}{1}\right)$$

$$PR(B) = 0.125 + 0.5\,PR(A), \qquad PR(C) = 0.125 + 0.5\,PR(B)$$

3단계 — 대입으로 푼다

$PR(B), PR(C)$를 $PR(A)$ 식에 대입:

$$PR(A) = 0.125 + 0.5\left(\underbrace{0.125 + 0.5\big(0.125 + 0.5\,PR(A)\big)}_{PR(C)} + \underbrace{0.125}_{PR(D)}\right) = 0.28125 + 0.125\,PR(A)$$

$$PR(A) = \frac{0.28125}{0.875} = \frac{9}{28} \approx 0.321$$

```
PR(B) = 0.125 + 0.5 * 0.321  # ≈ 0.286
PR(C) = 0.125 + 0.5 * 0.286  # ≈ 0.268
# 검산: 0.321 + 0.286 + 0.268 + 0.125 = 1.000
```

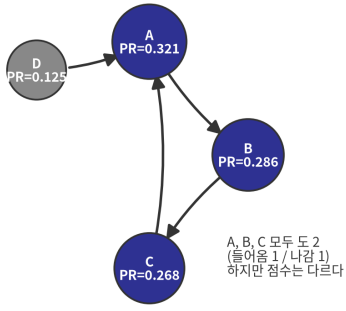
이 작은 예제에서 읽어둘 것 두 가지

- 겉모습은 똑같은데, 점수는 균등하지 않다

핵심

PageRank는 “엣지가 몇 개인가”가 아니라 “누가, 얼마나 중요한 위치에서 나를 가리키는가”의 문제.

$D \to A \to B \to C \to A$, $d = 0.5$ 손계산 결과

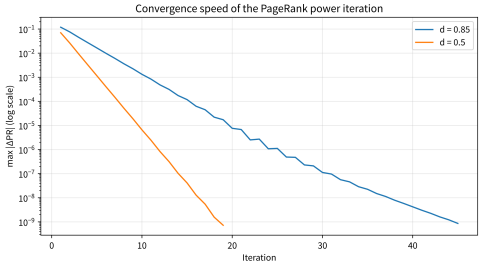


거듭제곱법이 왜 수렴하는가 — 바나흐 고정점 정리

- 이 거듭제곱법의 수렴 논리는, **ML2 Week 4**에서 벨만 최적방정식의 해가 유일하게 존재함을 보일 때 쓸 **바나흐 고정점 정리**(Banach fixed point theorem)와 **정확히 같은 논리**.
- "**어떤 변환을 값이 안 바뀔 때까지 반복해서 적용하면 결국 고정점**(fixed point)**에 도달한다**"는 같은 수학:
- 여기서 = "**그래프 위 확률분포**", 강화학습에서는 = "**상태의 가치함수**".

수렴이 얼마나 빠른가 — 지수적으로

- 매 반복의 변화량(전체 노드 중 최대 $|\Delta PR|$)은 **지수적으로** 줄어든다.
- 카라테 클럽: $d{=}0.85$ 일 때 대략 **45회**로 10^{-9} 이내 수렴. $d{=}0.5$ 이면 **19회**.
- d가 클수록 서퍼가 더 "**장기 기억**"을 하려 하므로 수렴이 **느려진다**.
- 작 중형 그래프: "**100회 반복**"이면 충분.
- 수십억 페이지 규모의 웹: 구글이 **분산 계산 · 희소화** 같은 별도 공학이 필요했던 이유.



매 반복의 최대 변화량 max $|\Delta PR|$ 을 로그 스케어로. $d{=}0.85$ (파랑)가 $d{=}0.5$ (주황)보다 더 오래 걸리며, 두 곡선 모두 직선(= 지수적 감소)을 따라 10^{-9} 아래로.

실무 1: “수렴했는지” 확인 없이 반복 횟수만 고정하기

```
def pagerank_converged(edges, n_nodes, d=0.85, tol=1e-9,
                       max_iters=1000):
    # ... 위( 와pagerank 동일, 단반복조건이다름   )
    pr = {i: 1 / n_nodes for i in range(n_nodes)}
    for _ in range(max_iters):
        new = {i: (1 - d) / n_nodes + d * sum(pr[j] / outdeg[j]
                 for j in incoming[i]) for i in range(n_nodes)}
        delta = max(abs(new[i] - pr[i]) for i in range(n_nodes))
        pr = new
        if delta < tol:
            break
    return pr
```

실무 2: dangling node (나가는 링크가 0인 노드)

- dangling 노드 존재 시 PR 합은 1
- 웹 그래프에서는 dangling 노드 보정이 표준

자주 하는 실수: “PageRank = 도(degree)”로 읽기

- 그러나 이 둘은 **같은 것이 아니다** — $\sum_{j \to i} \frac{PR(j)}{\text{outdeg}(j)}$ 는 **어떤** 노드가 나를 가리키느냐를 **가중**.

본래 설계 목적

방향 그래프에서는 차가 확 벌어진다: “중요하지 않은 페이지 10개가 링크”보다 “**가장 중요한 페이지 1개가 링크**”가 더 높은 PageRank를 만드는 것. **인기(도)**와 **권위**(가리켜지는 쪽의 중요도)는 다른 개념이며, PageRank가 측정하는 쪽은 **후자**.

지금 어디에 쓰이는가

- 방향 그래프 위 무작위 걷기의 정상분포 = 중요도
- 개인화 PageRank: 추천 시스템과 커뮤니티 감지

하나의 수학적 패턴

공유되는 패턴

PCA · word2vec · Node2Vec · PageRank는 얼핏 서로 다른 문제처럼 보이지만, 전부 **"무언가를 어떤 변환에 반복해서 통과시키면 특별한 지점(고유벡터, 임베딩, 정상분포)에 수렴한다"** 는 하나의 수학적 패턴을 공유한다.

지도 vs 비지도

지도학습이 "**정답을 맞히는 법**"을 배운다면, **비지도학습**은 "**데이터가 스스로 어떤 모양을 하고 있는지**"를 배운다 — 둘은 서로 다른 질문에 답한다.

확인 문제: 2-노드 그래프

2-노드 방향 그래프 $A \leftrightarrow B$ ($A \to B$, $B \to A$)에 $d = 0.5$, $N = 2$ 로 PageRank를 손으로 계산.

1. **단계 1** — 방정식 두 개: $PR(A) = 0.25 + 0.5\,PR(B)$, $PR(B) = 0.25 + 0.5\,PR(A)$
2. **단계 2** — 대칭성: $PR(A) = PR(B)$ 로 놓으면 $PR(A) = 0.25 + 0.5\,PR(A)$, 즉 $PR(A) = PR(B) = 0.5$ — d**의 값과 무관하게** 각 0.5. (코드로 확인)
3. **단계 3 (핵심)** — $B \to A$ 엣지를 지우면 (B는 **dangling node**가 됨): $PR(A) = 0.25$, $PR(B) = 0.25 + 0.5 \times 0.25 = 0.375$ — 합이 $0.25 + 0.375 =$ **0.625** 로, **1이 아니다**. **왜 그런지, 그리고 본문의 어떤 공학 보정을 추가하면 합이 1로 돌아올지 이름으로 답하라.** (힌트: “자주 하는 실수” 섹션의 두 번째 세부 사항)

자주 묻는 질문 (1/2)

- **Q. PageRank도 임베딩인가요?** — 아니요. Node2Vec은 각 노드를 **벡터 공간의 한 점**(임베딩)으로, PageRank는 각 노드에 **스칼라 값**(중요도 점수) 하나만. "구조적으로 얼마나 비슷한가"(Node2Vec) vs "전체에서 얼마나 중요한가"(PageRank).
- **Q. 댐핑 팩터 d가 없으면?** — $d=1$ 이면 순간이동이 전혀 없어 막다른 골목에서 서퍼가 **영원히 갇히거나**, 그래프가 여러 조각으로 나뉘어 있으면 **시작 조각에 따라 결과**가 달라짐. $d<1$ 의 순간이동 확률은 확률분포가 조각 사이를 계속 넘나들게 만들어 **유일한 해로 수렴**을 보장.
- **Q. 걷기 개수·길이는?** — 엄격한 공식은 없지만 원본 논문의 **노드당 10개 걷기, 길이 40, 차원 128**을 출발점으로 데이터 규모에 맞춰 줄인다(34개 노드에서는 10개 $\times$ 길이 8 · 차원 8이면 충분). 너무 짧으면 "문맥"이 바로 옆 이웃에 갇히고, 너무 길면 "출발 지점"을 잊어 문맥이 노이즈가 된다.

자주 묻는 질문 (2/2)

- **Q. 무방향 그래프에 써도 되나요?** — 된다. 표준 기법은 무방향 엣지 1개를 **방향 링크 2개**로 바꿔주는 것. 이렇게 하면 "도 순서에 가깝되, **누가 나를 가리키느냐를 가중한**" 순서가 되며, 일반적으로 도 순위와 동일하지는 않다.
- **Q. p, q는 그래프마다 튜닝해야 하나요?** — 보편적 최적값은 없다. 두 프리셋 $(1, 16), (16, 1)$도 "역할 구조 vs 커뮤니티 구조"에 맞춘 **출발점**에 불과 — 몇 가지 설정을 돌려보고 **downstream 태스크** 성능이나 시각화에서 기대된 구조가 보이나로 검증한다.

13.4 그래프 학습의 확장

두 아이디어 → 네 실전 방향

- 이 장에서 배운 두 아이디어: **무작위 걷기 기반 그래프 임베딩**(Node2Vec)과 **정상분포**(PageRank).

네 가지 실전 방향 (지도식 수업)

1. **(a) 단백질**: 구조를 알면, 그 구조로 기능을 분류
2. **(b) 금융**: 거래 그래프에서 자금세탁 탐지 (IBM AML World)
3. **(c) 교통**: 도로망 + 시계열의 스페이오템포럴 그래프 교통 예측(STGCN)
4. **(d) 추천**: 사용자-아이템 이분 그래프에서 개인화 PageRank/Node2Vec로 추천

(a) 단백질: 구조를 알면, 그 구조로 기능을 분류한다

- **DeepFRI: 구조에서 기능 분류**

자연스러운 파이프라인

구조 예측(AlphaFold) → 구조 기반 기능 분류(DeepFRI)



David Baker, Demis Hassabis, John Jumper — 2024년 노벨 화학상 컨퍼런스 (AlphaFold 개발 공로)

이 장의 도구가 어디에 쓰이는가

- 공간적 근접성으로 접촉 그래프 생성
- 그래프 컨볼루션 적용
- DeepFRI 기능 라벨 분류

왜 서열을 바로 분류하지 않고 구조 그래프를 거치는가

“왜?”를 한 번

동일한 기능을 가진 단백질은 **서열이 많이 달라**도 **3차원 형태(구조)는 비슷하게** 유지되는 경우가 흔하다.

- 구조 그래프는 “**서열과 무관한 기능의 공통 요인**”을 더 **직접적으로** 담는 표현.
- 같은 구조에 다른 서열의 단백질(**유사 동종단백질**) 기능을 추론할 때 **서열 기반 모델보다 유리**.

노트북 Part 1: 파이프라인의 뼈대

- 작은 **합성 접촉 그래프**(20개 노드)를 만들고.
- “**접촉 수(도)를 특징으로 한**” 매우 간단한 분류기를 돌린다.

주의: 이것은 DeepFRI의 GCN을 직접 학습하는 것이 아니다

“그래프 구조 특징 → 기능 분류” 파이프라인의 뼈대를 느끼는 데모.

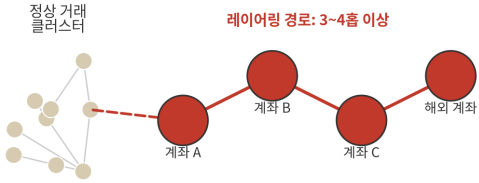
(b) 금융: 거래 그래프에서 자금세탁 탐지

- **IBM AML World**: **계좌를 노드, 거래를 엣지**로 하는 **대규모 합성** 거래 그래프 데이터셋.
- **그래프 구조 전체를 봐야 드러난다**

왜 합성 데이터셋인가 + 레이어링의 특징

합성인 것의 의미

“**답이 알려진** 거래 그래프”를 만들어 탐지 모델의 성능을 **측정**할 수 있게 하는 것 — 실제 금융 데이터는 **프라이버시** 때문에 공개가 어려우니, **구조와 노이즈 수준만 조절**해 “탐지 가능한 패턴”을 심어놓은 합성 데이터셋.



- 레이어링 패턴의 특징: 불법 자금의 흐름이 **1–2홉이 아니라 3–4홉 이상**으로 뻗어 있다.

이 장의 두 도구를 거래 그래프에

노드 임베딩 (Node2Vec 계열)

- 계좌를 “거래 네트워크에서 **어떤 위치에 있는가**”로 벡터화.
- 동일한 패턴으로 세탁에 쓰인 계좌들이 임베딩 공간에서 **같은 군집**에 모이면, 그 군집 자체가 “의심 패턴”의 **시그니처**.

중요도 점수 (PageRank 계열)

- “다른 계좌들이 **가장 많이 거쳐가는 통로**”에 높은 점수.
- 자금세탁의 **중계 계좌**(여러 출처를 여러 목적지로 합치고 흩어주는 허브) = 정확히 그런 **구조적 허브**.

두 신호를 결합하면

“무슨 특징인지 아직 이름이 붙지 않았지만” **구조적으로 수상한 계좌**를 선별 — 규칙 기반 탐지(“거래액 > 1억이면 신고”)가 놓치는 **패턴 전체가 정상인 개별 구간**까지 커버.

노트북 Part 2: 합성 거래 그래프

- 작은 **합성 거래 그래프**(20개 계좌, **정상 클러스터** + **의도적으로 심은 레이어링 경로**)를 만들고.
- **PageRank**로 “**중계 허브**”를 찾아보며.
- **1홉 특징만으로는 왜 발견 못 하는지** 보여준다.

(c) 교통: 도로망 위의 시계열 — spatio-temporal 그래프

- **도로망(그래프)은 고정**이지만, 각 도로 구간의 **교통량은 시간에 따라 변한다**.
- **STGCN**(Spatio-Temporal Graph Convolutional Network): **그래프 컨볼루션**(공간 방향: "이웃 도로의 정보를 합친다")와 **시계열 모델**(시간 방향: "과거 몇 스텝을 본다")을 결합.

주의

비슷한 계열의 **DCRNN**도 있는데, 그 논문의 "**diffusion convolution**"은 15장 생성모델의 diffusion과는 **다른 개념** — 여기서는 **그래프 위의 확산 과정**(무작위 걷기와 비슷한 발상)을 가리킨다.



캐나다 온타리오 401번 고속도로 정체 (2009) — 교통 예측 (STGCN) 응용의 실제 도로망 사례

왜 두 방향이 모두 필요한가

공간 방향만 — “이웃의 교통량을 합친다”

- **시간에 따른 변화**를 담을 수 없다: 출근 · 퇴근 시간대의 **파도**, 신호에 따른 **국소 정체**.

시간 방향만 — “이 도로의 과거 교통량만 본다”

- **다른 도로의 영향**을 놓친다: 인접 구간의 정체가 이 구간으로 **전파**.

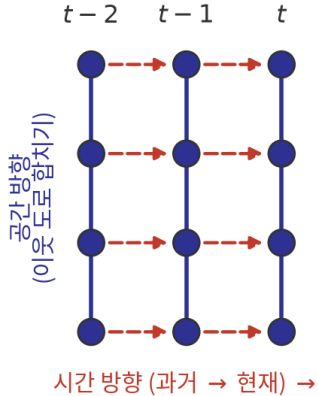
도로망은 그 **자신이 그래프**다 — 교차로가 노드이고 도로가 엣지.

STGCN: 공간 × 시간을 순환적으로 결합

1. 한 단계에서 (1) **공간 방향**으로 이웃 도로의 정보를 합치고,
2. (2) **시간 방향**으로 과거 몇 단계의 **자기 자신**을 본다.

- **인접 도로의 과거 흐름이 미래 흐름에 미치는 영향 학습**

STGCN: 매 단계마다 공간(파랑)+시간(빨강)을 함께 본다



노트북 Part 3: 합성 도로망

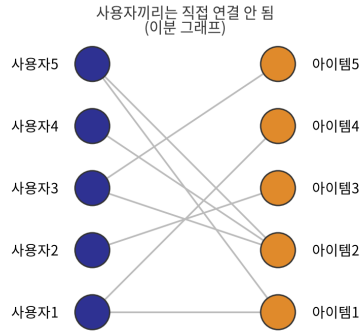
- 두 모델을 비교:
- (a) **1홉 이웃 평균**만 보는 모델
- (b) **3홉 이웃 평균 + 과거 3단계 자기 평균**을 보는 모델
- 결과: **이웃 범위를 넓히고 과거를 함께 보면 오차가 줄어든다**.

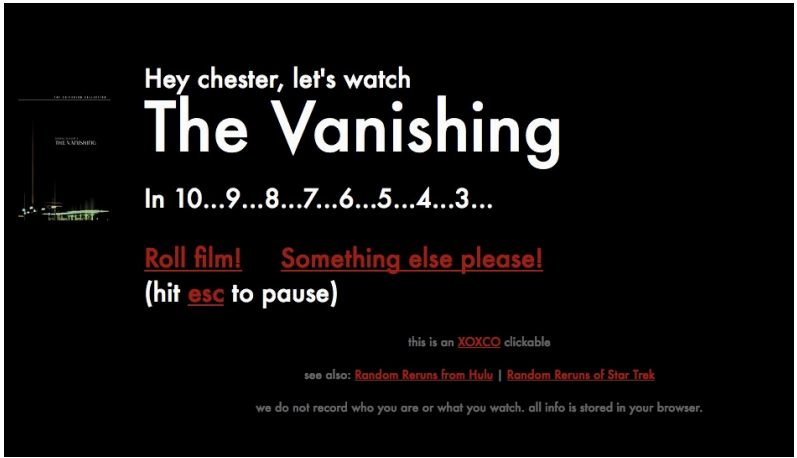
(d) 추천: 사용자-아이템도 그래프다

- **사용자-아이템 이분 그래프**

13.3절 개인화 PageRank가 이미 이 문제를 가리켰다

"무작위 걷기를 주어진 노드에서 다시 시작하는 변형 — 추천 시스템에 사용".





Netflix 추천 인터페이스 화면 (2010) — 사용자-아이템 이분 그래프가 실제로 서비스되는 모습

개인화 PageRank로 추천하기

- **개인화 PageRank: 사용자 근처 도달 빈도**

14장으로 이어지는 다리

같은 문제("사용자 × 아이템 행렬에서 빈 칸을 채운다")를 **행렬분해**(matrix factorization)로 풀면 — 놀랍게도 **PCA와 같은 수학**(14장에서 직접 확인).

네 사례를 한 장으로

사례	그래프(재료)	이 장의 도구	질문
(a) 단백질	잔기 **접촉 그래프**	**그래프 컨볼루션**(GNN)	“이 구조는 어떤 기능을 하는가?”
(b) 금융	**거래 그래프**	**노드 임베딩** + **중요도 점수**	“어떤 계좌가 구조적으로 수상한가?”
(c) 교통	**도로망** + 시계열	**공간** × **시간 반복 갱신**	“앞으로 N분 후의 교통량은?”
(d) 추천	사용자-아이템 **이분 그래프**	**개인화 PageRank** / **Node2Vec 임베딩**	“이 사용자가 다음에 뭘 원할까?”

재료가 그대로 실전에 옮겨진다

- **그래프가 도메인 지식에서 직접 만들어진다**
- **반복적 이웃 합산이 도구 핵심**

그래프 학습에서 “모델링”이 “프로그래밍”보다 중요한 이유

도메인이 달라질수록 **그래프를 어떻게 만드는가** (**어떤 것을 노드로, 어떤 것을 엣지로** 하는가)의 선택이 **모델의 성능을 결정**한다.

다음으로 가는 길: ML2 강화학습으로

- **MDP**는 "**상태를 노드로, 행동을 전환으로** 놓은 그래프"와 같은 구조를 갖고.
- Q-함수 / 가치함수는 그 위에서 "**반복해서 값을 갱신한다**"는 연산을 수행 (ML2 Week 4의 **벨만 방정식**).

13.4 → ML2

13장의 "**그래프 위에서 값을 갱신하는 감각**"이, ML2의 "**상태-행동 그래프 위에서 가치를 갱신하는 감각**"으로 이어진다.

이번 주차 정리

1. **13.1 무작위 걷기 = 마르코프 체인** — 전이확률 $P(i \to j) = 1/\deg(i)$, 무자기성, 정상분포 π^* 가 **도에 비례**. 걷기 말뭉치 $\to$ skip-gram $\to$ 카라테 두 파벌이 공간적으로 갈라짐.
2. **13.2 Node2Vec** — 걷기 경로 = 문장, 노드 = 단어로 skip-gram을 **그대로**. p, q = BFS형(역할) vs DFS형(커뮤니티) 다이얼. 실수: **무방향 대칭화 누락**.
3. **13.3 PageRank** — **같은 걷기에 다른 질문** (머무는 확률 = 중요도). 댐핑 + teleport, **거듭제곱법** (바나흐 고정점으로 수렴), 4-노드 손계산. $\neq$ **도**.
4. **13.4 확장** — 단백질(AlphaFold $\to$ DeepFRI) · 금융(AML World) · 교통(STGCN). "반복해서 이웃과 값을 합친다"가 실전 도구의 핵심.

다음 주 — Week 14. 표현학습

"함께 자주 나타나는 것은 가까운 벡터로 모인다"의 **텍스트** 버전 — **PCA**와 **word2vec**. 이번 장이 그래프 버전을 먼저 보여줬고, 14장에서 그 원형을 본문에서 배운다. ("반복해서 값을 갱신한다"는 감각은 ML2의 벨만 방정식으로 이어진다.)